Leetcode 371. Sum of Two Integers

April 2026

Understanding the Problem

This problem is relatively simple. We need to add two integers, a and b, but we can't use the + or - operators.

Solution

To start with this question, we need to understand what other tools we have at our disposal to manipulate integers other than traditional arithmetic operators. One way to do this is through **bitwise operators**. Before investigating how to add two whole integers, let's think about the addition process on a bit-by-bit level. Consider the following examples:

$$\begin{array}{r} 0 \\ +0 \\ \hline 0 \end{array} \qquad \begin{array}{r} 0 \\ +1 \\ \hline 1 \end{array} \qquad \begin{array}{r} 1 \\ +0 \\ \hline 1 \end{array} \qquad \begin{array}{r} 1 \\ +1 \\ \hline 1\mathbf{0} \end{array}$$

Pay attention to the last bit first. If the two bits have different values, the sum will have a final bit of 1. Otherwise, it will have a final bit of 0. The bitwise operator that has this property is XOR($\oplus$). In the case when there are two 1's, however, we need to introduce a carry bit onto the next bit. We can do this by initially having the carry bit set to 0 and change it to 1 if any two current bits (including the current carry bit) is 1. So, for each bit in the result:

$$a \oplus b \oplus \texttt{carry}$$

In order to save each intermediate result in the final answer, we will first need to initialize `result=0`. Then, we need to utilize the bitwise OR operator and a bit mask to save each intermediate value. This is how it will work:

$$\begin{array}{rr} mask & 1 \\ & \\ & 1 \\ + & 0 \\ \hline & 1 \end{array} \qquad \begin{array}{rrr} & 1 & 0 \\ & {\scriptstyle 1} & \\ & & 1 \\ + & & 1 \\ \hline & 1 & \mathbf{0} \end{array} \qquad \begin{array}{rrrr} & 1 & 0 & 0 \\ & {\scriptstyle 1} & & \\ & 0 & 1 & 0 \\ + & 0 & 1 & 1 \\ \hline & 1 & 0 & 1 \end{array}$$

To simulate this exact behavior in code, we can write the following for each position, $i+1$, in the final answer (`ans`) where $<<$ represents left bit shift. We are doing the left bit shift here to ensure that we are putting the bit in the right place.

$$\texttt{ans} = \texttt{ans} \mid (a \oplus b \oplus \texttt{carry} << i)$$

Handling Negative Integers

Since the constraints in this problem are $-1000 \leq a, b \leq 1000$, we need to ensure that we can add negative integers as well. Negative numbers are represented in binary with the 2's complement representation with the most significant bit representing the sign of the integer. Here is an example of 5 and -5:

$$\begin{array}{r} 5: \texttt{0000 0101} \\ -5: \underline{1}\texttt{111 1011} \\ \hline \texttt{10000 0000} \end{array}$$

Note that adding these integers together with the algorithm described earlier will result in 0, which is the expected answer since the last bit is excluded. However, what happens when the result is negative? Let's test it:

$$\begin{array}{r} 5: \texttt{0000 0101} \\ -10: \underline{1}\texttt{111 0110} \\ \hline \texttt{1111 1011} \end{array}$$

Note that this answer is the same as the 2's complement representation of -5. However, when this integer is returned, it will be 251 instead of -5. To identify the answer as negative, we need to turn the number into its positive representation again past the sign bit. To do this, we have to reverse the 2's complement process.

$$\begin{array}{r} \texttt{1111 1011} \\ \texttt{0000 0100} \\ \texttt{0000 0101} \end{array}$$

In order to do this programmatically without using addition, we can do the following:

```
return ~(ans ^ Integer.MAX_VALUE);
```